PROYECTO 5 · SPRINT 5

INTELLIGENT DOCUMENT ASSISTANT

DOCUMENTO DE ESTUDIO

Cómo se mide la recuperación

El sistema ya busca y responde. Ahora aprende a medir cada parte por separado: si los fragmentos correctos aparecen, qué tan arriba quedan y si la respuesta usa esos fragmentos sin inventar.

FICHA DE EVALUACIÓN #005

Documentos	**2 ficticios**
Chunks	**21**
Casos	**25 · 20 + 5**
Métricas	**Recall · P@k · MRR**
Estrategias	**3 base + rerank**
Fusión	**RRF · k = 60**
Rerank	**Opcional**
Total	**Sprint 5**

Medir antes de optimizar. Un número sin configuración no se puede reproducir.

TypeScript · PostgreSQL + pgvector · Vitest · OpenRouter

El reporte guarda modelo, parámetros, hashes y latencias.

RESUMEN EN 30 SEGUNDOS

Recuperar no es responder

El Sprint 4 conectó búsqueda y generación. El Sprint 5 separa las dos cosas para poder diagnosticarlas. Primero mide si la búsqueda recupera los fragmentos que contienen la respuesta. Después revisa, a mano, si el modelo usa esos fragmentos, los cita bien y se abstiene cuando no hay evidencia.

25	20 / 5	21	4
casos revisados a mano	respondibles / no respondibles	chunks ficticios con embeddings	estrategias comparables

> **La tesis del Sprint 5:** si no medís, no sabés si el problema está en la búsqueda, en el ranking, en el contexto o en el modelo. Las métricas de recuperación y la revisión de respuestas responden preguntas distintas y no se reemplazan entre sí.

El recorrido completo

GOLD SET

Preguntas con chunks relevantes conocidos

Cada caso declara el documento, los chunks que contienen la respuesta y si es respondible.

↓

RETRIEVAL

Cada estrategia devuelve un ranking

Vectorial, léxica, híbrida con RRF y, si está configurada, híbrida + rerank.

↓

MÉTRICAS

Recall@k, Precision@k y MRR

Se calculan por caso y se agregan solo sobre los casos respondibles.

↓

REPORTE

Un JSON reproducible

Modelo, revisión, topK, candidatos, RRF k, hashes, rangos y latencias.

↓

REVISIÓN MANUAL

Respuesta, fidelidad, citas y abstención

La búsqueda puede ser buena y la respuesta mala. Se revisan por separado.

La evaluación de recuperación no llama a un LLM. Solo usa el modelo local de embeddings y SQL. La revisión de respuestas sí usa el flujo de `/ask` del Sprint 4.

EL PROBLEMA

Dos preguntas, dos evaluaciones

Una búsqueda perfecta no garantiza una buena respuesta. Y una respuesta convincente puede estar construida sobre fragmentos irrelevantes. Por eso el sprint mide en dos capas.

CAPA	QUÉ PREGUNTA RESPONDE	CÓMO SE MIDE	HERRAMIENTA
Recuperación	¿Aparecen los chunks que contienen la respuesta? ¿En qué posición?	Recall@k, Precision@k, MRR.	`npm run eval:retrieval`
Respuesta	¿Usa esos chunks? ¿Los cita bien? ¿Se abstiene cuando no hay evidencia?	Revisión manual con hechos esperados y fuentes.	`eval/answer-quality-review.md`
Operación	¿Cuánto tarda y cuánto consume?	Latencia por caso, `usage` del proveedor.	Reporte + logs del servidor.

> **Un diagnóstico rápido:** si Recall@5 es bajo, el fragmento correcto ni siquiera llega a los candidatos. Si Recall es alto pero MRR es bajo, el fragmento aparece, pero demasiado abajo. Si la recuperación es buena y la respuesta falla, el problema está en el prompt, el modelo o el contexto.

Por qué no alcanza con “me parece que responde bien”

Una respuesta puede sonar segura y ser incorrecta. También puede ser correcta y citar una fuente que no sostiene la afirmación. La evaluación manual obliga a separar **exactitud**, **fidelidad**, **citas** y **abstención**, en lugar de dar una impresión general.

EXACTITUD

¿Está el hecho esperado?

Se compara contra los `expectedFacts` del gold set. No alcanza con que la respuesta sea plausible.

FIDELIDAD

¿Cada afirmación tiene respaldo?

Una afirmación importante sin fuente, o que contradice la fuente, es un fallo aunque el texto suene bien.

CITAS

¿El `[S#]` apunta al chunk correcto?

El servidor valida que el ID exista. La persona verifica que el excerpt sostenga la afirmación.

ABSTENCIÓN

¿Reconoce cuando no hay evidencia?

En un caso no respondible, inventar un dato es peor que decir “no encontré esa información”.

EL GOLD SET

Preguntas con respuesta conocida

El gold set es un conjunto pequeño de preguntas cuya respuesta y cuyos fragmentos relevantes conocemos de antemano. Sin etiquetas humanas, no hay forma de saber si una estrategia “mejoró” o simplemente devolvió otra cosa.

```
{
  "id": "q-02",
  "question": "¿Cuántos días de aviso exige el contrato para cancelar el servicio?",
  "documentId": "eval-doc-contrato",
  "relevantChunkIds": ["eval-chunk-contrato-04"],
  "answerable": true,
  "expectedFacts": ["Se requieren treinta (30) días corridos de aviso por escrito."]
}
```

Los casos no respondibles tienen la lista de chunks relevantes vacía. No se evalúan con Recall ni MRR: se usan para revisar la abstención.

```
{
  "id": "q-11",
  "question": "¿Cuál es el número de teléfono del propietario del edificio?",
  "documentId": "eval-doc-contrato",
  "relevantChunkIds": [],
  "answerable": false,
  "expectedFacts": []
}
```

Qué valida el cargador

- [ ] IDs de caso únicos.
- [ ] El documento referenciado existe en el fixture.
- [ ] Cada chunk relevante existe y pertenece a ese documento.
- [ ] `answerable: true` exige al menos un chunk relevante.
- [ ] `answerable: false` no puede declarar chunks relevantes ni hechos esperados.
- [ ] El fixture y el gold set no contienen documentos reales ni datos sensibles.

El fixture ficticio

DOCUMENTO	PÁGINAS	CHUNKS	QUÉ PERMITE PROBAR
`eval-doc-contrato`	2	12	Fechas, montos, plazos, códigos, preguntas no respondibles.
`eval-doc-manual`	2	9	Códigos de error, repuestos, especificaciones y garantía.

> **IDs deterministas:** el seed usa IDs como `eval-chunk-contrato-04` en lugar de cuids generados. Así el gold set puede referenciarlos de forma estable y el reporte se puede reproducir en otra máquina.

Cómo se siembra

```
npm run eval:seed
```

El comando reemplaza solo los documentos con IDs `eval-`, inserta páginas y chunks, y genera los embeddings con el mismo modelo local del resto de la app. Es idempotente: volver a ejecutarlo no duplica filas. Si editás el fixture o el gold set, hay que volver a sembrar antes de evaluar.

LAS MÉTRICAS

Recall, Precision y MRR

Las tres métricas miran el mismo ranking desde ángulos distintos. Juntas permiten distinguir “no aparece” de “aparece demasiado abajo” y de “aparece, pero hay mucho ruido”.

MÉTRICA	QUÉ MIDE	FÓRMULA	QUÉ DIAGNÓSTICO HABILITA
Recall@k	Proporción de chunks relevantes que aparecen en los primeros `k`.	`aciertos en top-k / total de relevantes`	Si es bajo, el fragmento correcto ni siquiera llega a los candidatos.
Precision@k	Proporción de los primeros `k` resultados que es relevante.	`aciertos en top-k / k`	Si es baja, el ranking trae ruido alrededor del acierto.
MRR	Qué tan arriba aparece el primer chunk relevante.	`1 / posición del primer relevante`	Si es bajo, el fragmento aparece, pero demasiado abajo.

Un ejemplo con números

Supongamos que los chunks relevantes son `chunk-12` y `chunk-13`, y el ranking devuelto es:

```
1. chunk-7
2. chunk-12
3. chunk-18
4. chunk-13
5. chunk-2
```

MÉTRICA	CÁLCULO	RESULTADO
Recall@5	2 relevantes encontrados / 2 relevantes totales	`1.0000`
Precision@5	2 relevantes en el top-5 / 5 posiciones	`0.4000`
MRR	El primer relevante está en la posición 2	`1 / 2 = 0.5000`

> **Precision@k divide por `k`, no por la cantidad devuelta.** Si una estrategia devuelve menos resultados, las posiciones ausentes cuentan como no relevantes. La regla se aplica a todas las estrategias y queda registrada en el reporte como `precisionAtKDenominator: "k"`. Sin esa regla, una lista corta podría inflar la precisión artificialmente.

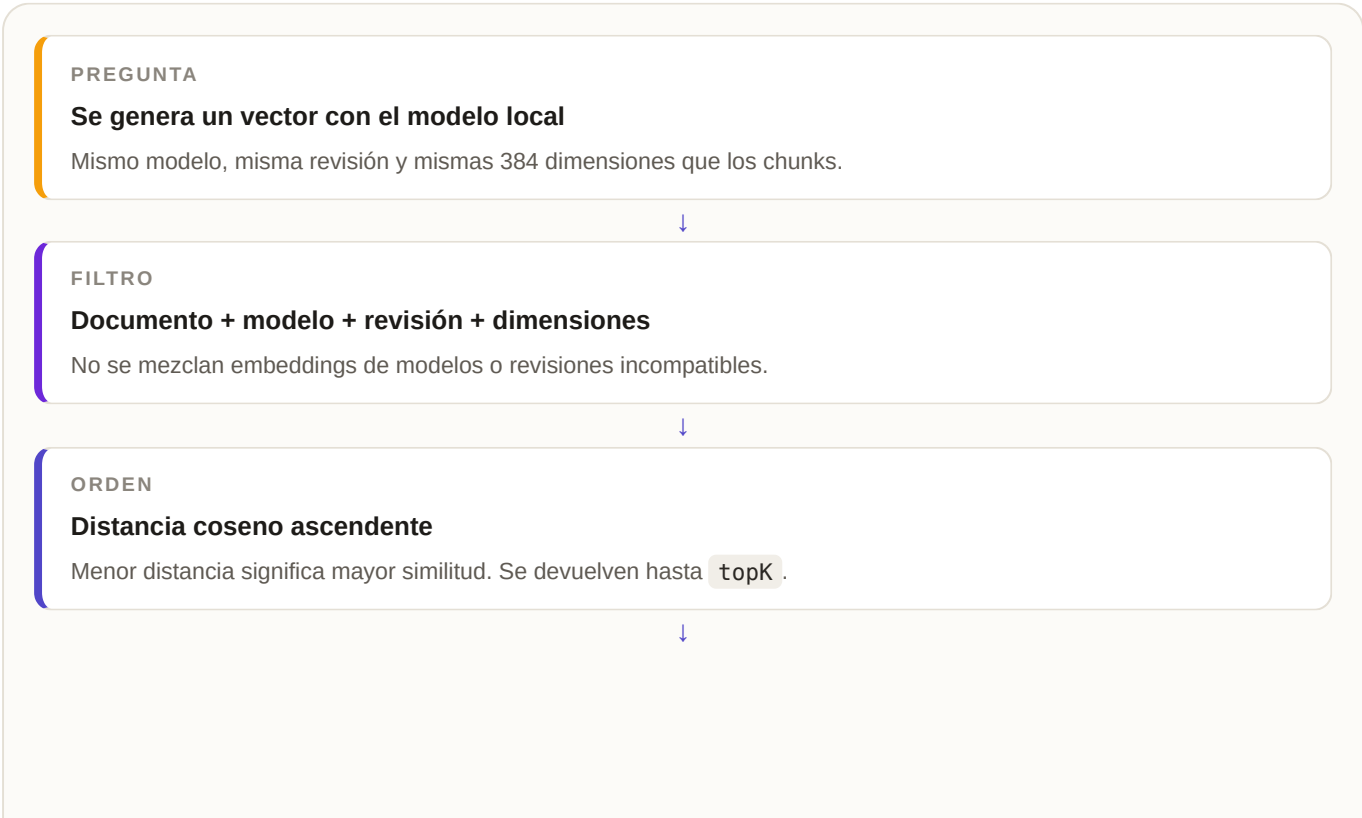
Qué hacer con cada número

SÍNTOMA	CAUSA PROBABLE	QUÉ INVESTIGAR PRIMERO
Recall@k bajo	El chunk correcto no está entre los candidatos.	Chunking, embeddings, filtro de documento, idioma, tamaño de la lista de candidatos.
Recall alto, MRR bajo	El chunk correcto aparece, pero abajo.	Ranking, RRF, reranker, tamaño de `k`.
Precision baja con Recall alto	Hay muchos resultados irrelevantes en el top-k.	Ranking y calibración; para respuestas, el presupuesto de contexto.
Recall y MRR altos, respuesta mala	La recuperación no es el cuello de botella.	Prompt, modelo, contexto, citas y abstención.

ESTRATEGIA A

Búsqueda vectorial

La búsqueda vectorial encuentra textos de significado parecido. El Sprint 5 no la reimplementa: reutiliza el embedding `query:` y la búsqueda exacta con el operador `<=>` de pgvector.



RESULTADO

Chunks con documento, página y contenido

La metadata queda disponible para las citas y para el reporte.

La app no tiene un modelo de usuarios: el único filtro existente es `documentId`. El Sprint 5 no inventó un filtro de usuario. El `topK` es configurable y la evaluación lo fija en 5 por defecto.

> **La búsqueda exacta es O(n)** sobre las filas compatibles. El corpus de evaluación es minúsculo, así que la latencia no representa un sistema en producción. Un índice ANN es trabajo futuro, no una mejora que este sprint pueda justificar con estos datos.

ESTRATEGIA B

Búsqueda léxica

La búsqueda de texto completo encuentra coincidencias de términos. Es especialmente útil para nombres, números, fechas y códigos exactos como `ESC-104`, `FLT-450-A` o `NA-2026-014`, donde la similitud semántica puede quedar corta.

tsvector y tsquery

PostgreSQL convierte el texto en un `tsvector` y la consulta en un `tsquery`. La configuración `simple` solo tokeniza y normaliza a minúsculas: no aplica stemming ni stopwords de un idioma particular.

```
SELECT
  c."id" AS "chunkId",
  ts_rank_cd(
    to_tsvector('simple', c."content"),
    websearch_to_tsquery('simple', $1)
  ) AS "lexicalScore"
FROM "DocumentChunk" c
JOIN "DocumentPage" p ON p."id" = c."documentPageId"
WHERE p."documentId" = $2
  AND to_tsvector('simple', c."content")
      @@ websearch_to_tsquery('simple', $1)
ORDER BY "lexicalScore" DESC, c."id" ASC
LIMIT $3;
```

El bug que encontró la evaluación

La primera versión pasaba la pregunta completa a `websearch_to_tsquery`. Ese operador une todos los términos con **AND**: para que un chunk coincidiera, tenía que contener todas las palabras de la pregunta. En la práctica, ningún chunk cumple eso. La evaluación real devolvió **cero resultados léxicos en los 25 casos**.

> **Moraleja:** una consulta que no falla no es lo mismo que una consulta que encuentra. El comando de evaluación detectó el problema porque comparó contra un gold set, no porque el SQL lanzara una excepción.

La corrección extrae los términos Unicode de la pregunta, conserva códigos con guiones, elimina duplicados, los pone entre comillas y los une con `or`. El texto sigue viajando como parámetro: nunca se concatena en el SQL.

ENTRADA	TSQUERY CONSTRUIDO	EFECTO
`¿Cuántos días de aviso?`	`"cuántos" or "días" or "de" or "aviso"`	Coincide con chunks que tengan cualquiera de los términos; el ranking premia los que tienen más.
`ESC-104`	`"esc-104"`	Conserva el código exacto y lo diferencia de una palabra genérica.
`...!!!`	`null`	No se ejecuta ninguna consulta a la base.

Después de la corrección, el Recall@5 léxico pasó de `0.0000` a `0.9500`. El MRR quedó más bajo que el vectorial (`0.6350`), lo que muestra que recuperar no es lo mismo que ordenar.

Índice GIN

```
CREATE INDEX "DocumentChunk_content_simple_fts_idx"
ON "DocumentChunk"
USING GIN (to_tsvector('simple', "content"));
```

La expresión del índice debe coincidir exactamente con la de la consulta para que PostgreSQL pueda usarlo. GIN acelera las búsquedas de texto completo a costa de almacenamiento y de escrituras más caras. En el corpus de 21 chunks el planner puede preferir un scan secuencial; con `enable_seqscan = off` se ve el `Bitmap Index Scan` sobre el índice. La migración es SQL manual porque Prisma no puede declarar índices de expresión en `schema.prisma`.

> **Prisma y los índices de expresión:** al crear una migración futura con `prisma migrate dev --create-only`, revisá el SQL generado. Prisma no conoce este índice y una migración automática podría intentar eliminarlo.

ESTRATEGIA C

Reciprocal Rank Fusion

La búsqueda híbrida combina la lista vectorial y la léxica. El problema es que sus scores no comparten escala: la distancia coseno es menor cuando el texto es más parecido, mientras que `ts_rank_cd` es mayor cuando hay más coincidencias. Sumarlos exigiría una normalización arbitraria.

RRF evita esa comparación y usa solo las **posiciones**:

```
RRF(d) = Σ 1 / (k + rank_s(d))
```

`k` es una constante que suaviza la importancia de las primeras posiciones. El valor por defecto es `60` y se configura con `RETRIEVAL_RRF_K`. Un chunk que aparece bien ubicado en ambas listas suma dos contribuciones; uno que aparece en una sola suma una.

Por qué no sumamos scores

FUENTE	SCORE	DIRECCIÓN	PROBLEMA DE SUMARLO
Vectorial	Distancia coseno `<=>`	Menor es mejor.	Un valor de 0.2 no significa lo mismo que 0.2 en la otra escala.
Léxica	`ts_rank_cd`	Mayor es mejor.	Depende de la cantidad de términos y de la densidad; no es una probabilidad.

Un ejemplo de fusión

Con `k = 60`, un chunk en la posición 2 de la lista vectorial y en la posición 1 de la léxica suma:

```
1 / (60 + 2) + 1 / (60 + 1)
= 1/62 + 1/61
≈ 0.016129 + 0.016393
≈ 0.032522
```

CHUNK	RANK VECTORIAL	RANK LÉXICO	SCORE RRF	RESULTADO
`chunk-b`	2	1	`0.032522`	Sube por aparecer bien en ambas listas.
`chunk-a`	1	—	`0.016393`	Solo compite con una fuente.
`chunk-c`	—	2	`0.016129`	Solo compite con la otra.

> **Duplicados:** dentro de una misma lista, un chunk repetido se cuenta una sola vez. Si aparece en ambas listas, se acumulan exactamente dos contribuciones, una por fuente. El resultado conserva `sourceRanks` para poder rastrear de dónde salió cada posición. Los empates se resuelven por mejor rank y después por `chunkId`, para que el reporte sea determinista.

La evaluación pide 20 candidatos a cada fuente, fusiona y devuelve el top-5. Así el mismo caso se puede comparar contra el baseline vectorial sin cambiar el gold set.

ESTRATEGIA D

Reranking opcional

El reranker recibe la pregunta y una lista chica de candidatos, y vuelve a ordenarlos comparando la pregunta con cada texto. No reemplaza a la búsqueda: la mejora después de recuperar.

```
Búsqueda vectorial: primeros 20
Búsqueda léxica:    primeros 20
RRF:                mejores 20 combinados
Reranker opcional:  mejores 5
LLM:                responde con esos 5
```

El contrato

```
interface Reranker {
  readonly model: string;
  rerank(input: {
    query: string;
    candidates: readonly RerankCandidate[];
    topN: number;
  }): Promise<RerankOutcome>;
}
```

El cliente de OpenRouter usa `POST /api/v1/rerank`. El body lleva `model`, `query`, `documents` y `top_n`. La respuesta trae `results` con `index` y `relevance_score`, más un `usage` con `search_units`, `total_tokens` y, a veces, `cost`.

Qué se valida

- [ ] El índice devuelto es un entero dentro del rango de candidatos enviados.
- [ ] Ningún índice aparece dos veces.
- [ ] El score de relevancia es finito.
- [ ] La respuesta vacía se rechaza si había candidatos.
- [ ] Timeout, 429, 5xx, error de red y JSON malformado se convierten en errores seguros.
- [ ] La latencia y el usage se registran sin guardar la pregunta ni el texto de los candidatos.

> **Datos externos:** al activar el reranking, la pregunta y los chunks candidatos se envían a OpenRouter. Usá solo el fixture ficticio o documentos que puedas compartir. El reranking está desactivado si `RERANK_MODEL` está vacío o no hay clave; el comando normal no falla por no tenerlo.

No se asume que exista un modelo gratuito. El modelo y su precio pueden cambiar. Si no hay un modelo configurado, la estrategia se reporta como `skipped` con el motivo `missing_model` o `disabled_by_flag`.

LA HERRAMIENTA

Un comando, un reporte

La evaluación se ejecuta desde la raíz del proyecto, sin llamar a un LLM. El comando imprime un resumen y escribe un JSON con todo lo necesario para reproducir la comparación.

```
npm run eval:seed          # una vez, o después de editar el fixture
npm run eval:retrieval     # ejecuta los 25 casos
npm run eval:retrieval -- --top-k 5 --candidates 20 --rrf-k 60
npm run eval:retrieval -- --no-rerank
```

FLAG	DEFAULT	QUÉ HACE
`--top-k`	`EVAL_TOP_K` o 5	Corte de Recall@k, Precision@k y MRR.
`--candidates`	`RETRIEVAL_CANDIDATES` o 20	Candidatos que devuelve cada fuente antes de RRF.
`--rrf-k`	`RETRIEVAL_RRF_K` o 60	Constante de Reciprocal Rank Fusion.
`--no-rerank`	—	Omite el reranking aunque haya un modelo configurado.
`--out`	`eval/reports/retrieval-<fecha>.json`	Ruta del reporte.

Qué guarda el reporte

- [ ] Fecha y versión de Node.
- [ ] Rutas y hashes SHA-256 del fixture y del gold set.
- [ ] Modelo, revisión y dimensiones del embedding.
- [ ] `topK`, candidatos y constante RRF.
- [ ] Estado del reranking: modelo, `topN`, motivo de omisión, latencia y usage.
- [ ] Agregados por estrategia: Recall, Precision, MRR, latencia media y casos fallidos.
- [ ] Por caso: IDs recuperados, rank de cada chunk relevante, métricas y latencia.
- [ ] Notas sobre el denominador de Precision, la latencia del embedding y el envío externo del rerank.

> **La latencia no se mezcla:** el embedding de la pregunta se mide una vez por caso y se informa como `meanEmbeddingMs`. La latencia de cada estrategia mide la búsqueda y la fusión, no vuelve a contar el embedding.

LA EJECUCIÓN REAL

Qué dijeron los números

Esta es la ejecución de referencia del 12 de noviembre de 2026, con 25 casos, `topK = 5`, 20 candidatos, RRF `k = 60` y el modelo local E5-small. No es un resultado universal: es una foto reproducible de este corpus.

ESTRATEGIA	RECALL@5	PRECISION@5	MRR	LATENCIA MEDIA
`vector`	1.0000	0.2100	0.9667	2–3 ms
`lexical`	0.9500	0.2000	0.6350	2–3 ms
`hybrid`	1.0000	0.2100	0.9000	2–3 ms
`hybrid-rerank`	—	—	—	skipped (`missing_model`)

Lectura honesta

- [ ] El corpus es chico y los documentos fueron escritos para ser recuperables. No generalices el 1.0000 de Recall.
- [ ] Precision@5 cercano a `0.21` no significa “mal retrieval”: con un chunk relevante y `k = 5`, el máximo por caso es `0.2`. El caso `q-06` tiene dos chunks relevantes y eleva el promedio.
- [ ] La léxica recupera el 95 % de los relevantes, pero los ordena peor. Es útil para códigos exactos, no para reemplazar al vector.
- [ ] La híbrida mantiene Recall 1.0 pero baja el MRR porque RRF también incorpora las posiciones léxicas. En este dataset el baseline vectorial ya era muy fuerte.
- [ ] El reranking no se ejecutó: no hay `RERANK_MODEL` configurado. El cliente se validó con `fetch` mockeado.

La prueba de sensibilidad

Si las métricas no cambian al mover el corte, algo está mal en el cálculo. Con `--top-k 1`:

ESTRATEGIA	RECALL@1	PRECISION@1	MRR
`vector`	0.9250	0.9500	0.9500
`lexical`	0.4250	0.4500	0.4500
`hybrid`	0.8250	0.8500	0.8500

> **No declares un ganador universal con 20 casos.** Una diferencia de MRR puede ser ruido. La comparación sirve para ver el efecto de cada estrategia y para detectar regresiones, no para afirmar qué técnica es mejor en todos los corpus.

LA RESPUESTA

Evaluar sin LLM juez

En este sprint la calidad de respuesta se revisa a mano. Un juez automático puede servir más adelante, pero primero hace falta una muestra revisada que sirva de referencia y un acuerdo explícito sobre los criterios.

ASPECTO	QUÉ SE COMPRUEBA	RESULTADO
Exactitud	¿La respuesta contiene los `expectedFacts` sin contradecirlos?	`sí` / `parcial` / `no`
Fidelidad	¿Cada afirmación importante está respaldada por una fuente recuperada?	`respaldada` / `parcial` / `sin respaldo`
Citas	¿Cada `[S#]` apunta a una fuente recuperada y su excerpt sostiene la afirmación?	`válida` / `inválida`
Abstención	En un caso no respondible, ¿reconoce que no hay evidencia?	`correcta` / `incorrecta` / `n/a`
Latencia y uso	¿Cuánto tardó y cuántos tokens consumió?	Segundos y `usage` del proveedor

El caso que hay que mirar con cuidado

`q-11` pregunta por el teléfono del propietario de un edificio. El contrato tiene un teléfono de soporte, pero no es ese dato. La respuesta correcta es abstenerse. Si el modelo devuelve el teléfono de soporte como si fuera el del propietario, la cita puede ser válida —el chunk existe— pero la afirmación no responde la pregunta.

> **No confundas `noEvidence` con abstención.** La app no tiene umbral de similitud: la búsqueda vectorial siempre devuelve top-5. En una pregunta no respondible, el modelo recibe chunks aunque no sean relevantes. La abstención se evalúa en el texto de la respuesta, no en el flag `noEvidence`, que solo cubre “no hay embeddings” o “el contexto quedó vacío”.

Cómo se llena

- [ ] Ejecutá `npm run eval:seed` y levantá `npm run dev`.
- [ ] Abrí `/ask` y elegí el documento del caso.
- [ ] Pegá la pregunta del gold set y guardá la respuesta completa.
- [ ] Marcá exactitud, fidelidad, citas y abstención en la plantilla.
- [ ] Anotá latencia de retrieval/generación y tokens de `usage`.
- [ ] No cambies el prompt durante la revisión.

La plantilla está en `eval/answer-quality-review.md`. La revisión de las 25 respuestas es un paso manual pendiente: requiere llamadas reales al proveedor y no se ejecutó dentro de este sprint para no generar un costo que nadie pidió.

LOS LÍMITES

Lo que este sprint no resuelve

LÍMITE	POR QUÉ IMPORTA	QUÉ HARÍA FALTA
25 casos, 2 documentos ficticios, un modelo de embeddings	Es una muestra para aprender y detectar regresiones, no un benchmark representativo.	Ampliar el gold set y sumar documentos reales anonimizados.
Sin stemming español/italiano	`simple` no reduce palabras a su raíz; “atención” no coincide con “atiende”.	Probar configuraciones lingüísticas y medir el efecto.
Sin índice ANN	La búsqueda vectorial exacta es O(n); la latencia no representa producción.	Medir HNSW/IVFFlat con un corpus grande.
Sin juez automático	La revisión manual no escala y puede ser subjetiva.	Comparar un juez LLM contra la revisión humana, sin reemplazarla.
Sin umbral de similitud	La búsqueda siempre devuelve top-k, incluso si nada es relevante.	Calibrar un umbral con datos, no a ojo.
Rerank real no ejecutado	No hay un modelo configurado ni se asumió un modelo gratuito.	Configurar un modelo, medir costo y comparar antes/después.
Revisión de respuestas pendiente	La plantilla existe, pero no se revisaron las 25 respuestas.	Ejecutar `/ask` con los casos y completar la tabla.

Próximos pasos

- [ ] Ampliar el gold set y agregar documentos reales anonimizados.
- [ ] Conectar la estrategia ganadora a `/ask` con una comparación A/B.
- [ ] Hacer la revisión manual de respuestas y citas.
- [ ] Evaluar configuraciones de texto en español e italiano.
- [ ] Medir costo y latencia con más volumen.
- [ ] Convertir `eval:retrieval` en una comprobación de regresión repetible.
- [ ] Recién después decidir umbral, reranking pago o índice ANN.

MANOS A LA OBRA

Cómo probarlo

Variables de entorno

VARIABLE	DEFAULT	PARA QUÉ
`EVAL_TOP_K`	5	Corte de las métricas.
`RETRIEVAL_CANDIDATES`	20	Candidatos por fuente antes de RRF.
`RETRIEVAL_RRF_K`	60	Constante de RRF.

VARIABLE	DEFAULT	PARA QUÉ
`RERANK_MODEL`	vacío	Modelo de reranking; vacío lo desactiva.
`RERANK_API_KEY`	vacío	Clave opcional; si falta, usa `OPENROUTER_API_KEY`.
`RERANK_TOP_N`	5	Candidatos que devuelve el reranker.
`RERANK_TIMEOUT_MS`	20000	Timeout del reranker.

Comandos

```
npm run eval:seed
npm run eval:retrieval
npm run eval:retrieval -- --top-k 1
npm run eval:retrieval -- --rrf-k 1
npm run eval:retrieval -- --no-rerank
npm test
```

Pruebas manuales

- [ ] Verificar que `eval:seed` deja 2 documentos y 21 chunks.
- [ ] Confirmar que el reporte incluye modelo, revisión, topK, candidatos y RRF k.
- [ ] Cambiar `--top-k 1` y comprobar que Recall y MRR bajan.
- [ ] Comparar vector y léxico en un caso con código exacto como `ESC-104`.
- [ ] Revisar que una consulta léxica vacía no toque la base.
- [ ] Configurar el rerank y comparar el orden antes y después.
- [ ] Revisar una respuesta y comprobar cada cita contra su excerpt.
- [ ] Probar un caso no respondible y confirmar la abstención.

Para revisar el código

1. `eval/gold-set.json` y `eval/fixtures/fictional-documents.json`: qué se considera relevante.
2. `src/lib/evaluation/dataset.ts`: validación Zod e integridad referencial.
3. `src/lib/evaluation/metrics.ts`: Recall, Precision con denominador `k` y MRR.
4. `src/lib/retrieval/lexical.ts`: tsvector, tsquery OR y SQL parametrizado.
5. `src/lib/retrieval/rrf.ts`: fusión por posiciones, duplicados y desempate.
6. `src/lib/retrieval/hybrid.ts`: ventana de candidatos.
7. `src/lib/retrieval/reranker.ts` y `openrouter-reranker.ts`: validación y cliente opcional.
8. `src/lib/evaluation/run.ts`: orquestación, errores y agregados.
9. `scripts/eval-retrieval.ts` y `scripts/eval-seed.ts`: CLI, reporte y fixture.
10. `prisma/migrations/20261112220247_add_document_chunk_fts_index/migration.sql`: índice GIN.

REPASO

Ocho preguntas

1. ¿Qué diferencia hay entre evaluar la recuperación y evaluar la respuesta generada?

La recuperación se evalúa con rankings y chunks relevantes: Recall, Precision y MRR. La respuesta se evalúa con exactitud, fidelidad, citas y abstención. Una buena recuperación no garantiza una buena respuesta, y una respuesta correcta puede estar basada en evidencia incompleta.

2. Si Recall@5 es bajo, ¿qué parte del pipeline conviene investigar primero?

Primero la recuperación: chunking, embeddings, filtro de documento, idioma y tamaño de la lista de candidatos. Si el chunk correcto no aparece, el ranking y el reranker no tienen nada que reordenar.

3. ¿Por qué la búsqueda léxica puede encontrar un código exacto que la vectorial no prioriza?

Porque el código es un término literal. La búsqueda vectorial representa significado y puede diluir un código poco frecuente entre textos semánticamente parecidos; la léxica lo busca como token y lo rankea por coincidencia.

4. ¿Qué combina RRF: los scores originales o las posiciones de los resultados?

Las posiciones. RRF usa `1 / (k + rank)` para cada lista y suma las contribuciones. No compara distancia coseno con `ts_rank_cd` porque no comparten escala ni significado.

5. ¿Qué agrega el reranker y qué costo puede introducir?

Vuelve a ordenar un conjunto chico de candidatos comparando la pregunta con cada texto. Agrega una llamada externa, latencia, posible costo por uso y el riesgo de enviar la pregunta y los fragmentos a un proveedor. Por eso es opcional y está desactivado sin configuración.

6. ¿Por qué necesitamos etiquetar manualmente qué chunks son relevantes?

Porque sin ground truth no hay forma de saber si una estrategia mejoró o solo devolvió otra cosa. Las etiquetas humanas permiten calcular Recall y MRR y revisar la abstención con un criterio independiente del modelo que se evalúa.

7. ¿Por qué Precision@k divide por `k` y no por la cantidad devuelta?

Para que todas las estrategias se comparen con el mismo denominador. Si una devuelve menos resultados, las posiciones ausentes cuentan como no relevantes. Si se dividiera por lo devuelto, una lista corta podría parecer artificialmente precisa.

8. ¿Qué significa que el reporte guarde hashes, modelo y parámetros?

Que la comparación se puede reproducir. Si cambia el fixture, el gold set, el modelo o `topK`, el reporte lo delata. Un número sin esa metadata no sirve para comparar dos ejecuciones.

GLOSARIO

Términos para tener a mano

Gold set

Conjunto de preguntas con chunks relevantes y hechos esperados, revisado a mano. Es la referencia de la evaluación.

Fixture

Documentos y chunks ficticios que se cargan en la base para que el gold set tenga IDs estables.

Chunk

Fragmento de una página. Es la unidad que se vectoriza, se recupera y se cita.

Precision@k

Proporción de los primeros `k` resultados que es relevante. En este proyecto divide por `k`.

RRF

Reciprocal Rank Fusion. Combina rankings por posición, no por score.

Búsqueda híbrida

Combinación de la lista vectorial y la léxica. En este sprint se fusiona con RRF.

Abstención

Respuesta que reconoce que las fuentes no contienen información suficiente. Es distinta de “no se recuperaron chunks”.

Cita válida

Un `[S#]` que apunta a una fuente recuperada y cuyo excerpt sostiene la afirmación citada.

Configuración simple

Configuración de texto de PostgreSQL que tokeniza y baja a minúsculas sin stemming ni stopwords de un idioma.

Recall@k

Proporción de chunks relevantes que aparecen entre los primeros `k` resultados.

MRR

Reciprocal rank del primer resultado relevante. Si aparece en la posición 2, aporta `1/2`.

Búsqueda léxica

Búsqueda por coincidencia de términos con `tsvector` y `tsquery`.

Reranker

Modelo que recibe la pregunta y candidatos y los vuelve a ordenar. Es opcional en este sprint.

Fidelidad

Grado en que las afirmaciones de la respuesta están respaldadas por las fuentes recuperadas.

Índice GIN

Índice invertido de PostgreSQL que acelera búsquedas de texto completo. Agrega costo de almacenamiento y escritura.

Reporte reproducible

JSON con fecha, modelo, parámetros, hashes y resultados por caso. Permite comparar dos ejecuciones.

Checklist para el Sprint 6

- [ ] Ampliar el gold set y sumar documentos reales anonimizados.
- [ ] Hacer la revisión manual de respuestas y citas.
- [ ] Conectar la estrategia ganadora a `/ask` con una comparación A/B.
- [ ] Probar configuraciones de texto en español e italiano.
- [ ] Medir costo y latencia con más volumen.
- [ ] Evaluar un juez LLM contra la revisión manual.
- [ ] Convertir la evaluación en una regresión repetible.
- [ ] Recién después decidir umbral, reranking pago o índice ANN.

> **La frase para llevarte:** el Sprint 1 construyó el archivo. El Sprint 2 lo cortó en fichas. El Sprint 3 le dio a cada ficha una coordenada. El Sprint 4 le pidió al modelo que redacte usando solo esas fichas. El Sprint 5 aprendió a medir si las fichas correctas aparecen, si el orden sirve y si la respuesta se queda dentro de la evidencia. Sin medición, “mejorar” es solo cambiar cosas.